Indian Institute of Technology Gandhinagar

Department of Computer Science and Engineering

CS 432 – Databases

Assignment 3 – Course Project

Module A

Advanced Transaction Engine & Crash Recovery

Project: ACID properties on B+ Trees

GitHub:
https://github.com/AnuragSingh0000/DB_Assignment_3

Video: Link

Team Member	Roll Number
Tanish Yelgoe	23110328
Srivaths P	23110321
Venkatakrishnan E	23110357
Anurag Singh	23110035
Arjun B Dikshit	23110040

Instructor: Dr. Yogesh K. Meena

Course: CS 432 – Databases (Track 1)

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.

Contents

1 ACID Validation: Correctness of Operations **2**

1.1 Overview . . . 2

1.2 Implementation . . . 2

1.2.1 Atomicity: Undo Logging and Automatic Rollback . . . 2

1.2.2 Consistency: Constraint Validation and Referential Integrity . . . 4

1.2.3 Failure Handling . . . 5

1.2.4 Multi-User Conflict Handling . . . 6

1.3 Testing . . . 7

1.3.1 Atomicity Tests . . . 8

1.3.2 Consistency Tests . . . 9

1.3.3 Durability Tests . . . 9

1.3.4 Isolation Tests . . . 10

1.4 Observations and Limitations . . . 11

1.4.1 Observations . . . 11

1.4.2 Limitations . . . 11

Chapter 1

ACID Validation: Correctness of Operations

1.1 Overview

This module validates that all database operations built on top of the custom B+ Tree storage engine satisfy full ACID guarantees. The implementation is a Sports Management database (`sports_db`) with four interrelated tables (`member`, `sport`, `team`, and `equipment` which are seeded with 20 members, 8 sports, 15 teams, and 15 equipment items). The core component responsible for enforcing ACID is the `TransactionManager` in `transaction_manager.py`, which wraps every mutation to the underlying `Table` (and its internal `BPlusTree`) behind a transactional interface backed by a Write-Ahead Log (WAL).

The architecture consists of four collaborating classes: `WALManager` handles durable append-only logging with `fsync`; `LockManager` enforces per-table exclusive locks; `Transaction` tracks the undo log and acquired locks for a single unit of work; and `TransactionManager` orchestrates all four ACID properties by composing these components.

1.2 Implementation

1.2.1 Atomicity: Undo Logging and Automatic Rollback

Atomicity is enforced through an in-memory undo log maintained inside each `Transaction` object, mirrored by the WAL on disk. Every CRUD wrapper (`insert`, `update`, `delete`) follows the same strict protocol:

1. Acquire an exclusive lock on the relevant table(s) to guarantee isolation.
2. Validate foreign key constraints *before* touching the tree.
3. Call `tx.record_insert / record_update / record_delete`, which writes a `before`/`after` snapshot to both the in-memory `_ops` list and the WAL via `WALManager.log_operation()`.

4. Apply the change to the `Table` (and therefore to the B+ Tree).
5. If *any* exception is raised at any step, the `except` block immediately calls `self.rollback(tx)`.

Acquiring locks is critical for atomicity as it ensures the state of the target records cannot mutate between the snapshot phase and the execution phase.

The `rollback()` method iterates `tx.undo_ops()`, which returns the `_ops` list in *reverse chronological order*, and calls `_undo_single()` on each entry. The undo logic is symmetric: an INSERT is reversed by `table.delete(key)`, a DELETE is reversed by `table.insert(before)`, and an UPDATE is reversed by restoring the `before` record.

For the special case of a Primary Key–shifting update (where `before[search_key] != after[search_key]`), the undo step deletes the new PK and re-inserts the old record, ensuring the B+ Tree is left in exactly its pre-transaction state. Furthermore, if an error occurs during the rollback loop itself, the engine catches the exception and proceeds to reverse the remaining operations, maximizing resilience.

```
def _undo_single(self, op_entry):
    op, key = op_entry["op"], op_entry["key"]

    if op == OpType.INSERT:
        table.delete(key)
    elif op == OpType.DELETE:
        table.insert(op_entry["before"])
    elif op == OpType.UPDATE:
        old_record = op_entry["before"]
        old_pk = old_record[table.search_key]
        new_record = op_entry.get("after")

        if new_record and new_record[table.search_key] != old_pk:
            table.delete(new_record[table.search_key])
            table.insert(old_record)
        else:
            table.update(old_pk, old_record)
```

Because every operation records its `before` image *prior* to execution, the undo log always has enough information to reverse any partial sequence of changes, regardless of how many tables were touched.

1.2.2 Consistency: Constraint Validation and Referential Integrity

Consistency is enforced at two layers: the `Table` class and the `TransactionManager`.

Table-level constraints. `Table.validate_record()` is called before every insert or update. It checks type correctness, `NOT NULL` rules, `UNIQUE` constraints (by scanning the B+ Tree via `get_all()`), and `CHECK` expressions. CHECK rules are stored as a list of plain strings (e.g., `'x >= 0'`) and evaluated at runtime using Python's `eval()`, with the column value bound to `x`. This design allows constraints to survive pickling to `database.dat` without using non-serialisable lambdas.

Forward Foreign Key checks (child → parent). Before any insert or update, the `TransactionManager` inspects `table.foreign_keys` which is a dictionary mapping local column names to the referenced table names, and verifies that each referenced PK exists in the parent B+ Tree. A missing value in the parent (i.e referencing value does not exist in the referenced column) raises a `ConstraintError`, which triggers an automatic rollback before the child record is written.

Reverse Foreign Key checks (parent → child). Before a delete or a PK-changing update on a *parent* table, the manager inspects `table.referenced_by` a list of `(child_table, child_column)` tuples. It scans each child table's B+ Tree and raises a `ConstraintError` if any child record still points to the parent PK being removed or changed. This prevents dangling foreign key references (orphaned rows) from ever reaching a committed state.

```
# Reverse FK check in delete()
for child_table_name, child_col in table.referenced_by:
    child_table = self._get_table(db_name, child_table_name)
    for _, child_rec in child_table.get_all():
        if child_rec.get(child_col) == record_id:
            raise ConstraintError(
                f"Cannot delete '{record_id}' from '{table_name}'. "
                f"It is still referenced by '{child_table_name}'."
            )
```

Because both the forward and reverse checks happen *inside* the transaction before any B+ Tree mutation, a constraint violation always results in a clean rollback with no partial data written.

1.2.3 Failure Handling

In-Transaction Failures: Undo Logging and Automatic Rollback

Every CRUD wrapper (`insert`, `update`, `delete`) wraps its body in a `try/except` block. Any exception (whether a `ConstraintError`, a `TransactionError`, or an unexpected runtime error) immediately calls `self.rollback(tx)` before re-raising to the caller. This means no external cleanup code is required; a failed transaction always cleans up after itself.

The rollback mechanism relies on an in-memory undo log inside each `Transaction` object. Each operation, before touching the B+ Tree, calls one of `record_insert`, `record_update`, or `record_delete`, which appends a `{before, after}` snapshot to the `_ops` list and simultaneously writes an `OP` entry to the WAL. `rollback()` then calls `tx.undo_ops()`, which returns the list in *reverse chronological order*, and applies `_undo_single()` to each entry:

```
if op == OpType.INSERT:
    table.delete(key)
elif op == OpType.DELETE:
    table.insert(op_entry["before"])
elif op == OpType.UPDATE:
    old_pk = old_record[table.search_key]
    if new_record and new_record[table.search_key] != old_pk:
        # PK-shifting update: delete the new key, restore the old
        table.delete(new_pk)
        table.insert(old_record)
    else:
        table.update(old_pk, old_record)
```

Because the `before` image is captured *before* execution, the undo log has sufficient information to reverse any partial sequence of changes regardless of how many tables or operations were involved. The PK-shifting case requires special handling: since the B+ Tree stores records under their key, a key change means the old key no longer exists and the new key must be deleted before the old record can be restored.

Crash Recovery: Two-Phase WAL Replay

On startup, `TransactionManager.__init__()` calls `load_from_disk()` to restore the last clean snapshot from `database.dat`, then immediately calls `recover()` to replay the WAL. The recovery procedure handles two distinct crash scenarios.

Phase 1: REDO committed transactions. For every transaction whose WAL contains a `COMMIT` record, all operations are replayed forward in chronological order using `_redo_single()`. This covers the gap between the WAL write and the `save_to_disk()` call: a transaction may have committed and written its `COMMIT` to the WAL, but the crash occurred before the snapshot was flushed to disk. The REDO phase re-applies those operations idempotently i.e. it checks whether a key already exists before inserting, and whether it is absent before deleting, so replaying a WAL produces no duplicates.

Phase 2: UNDO incomplete transactions. For every transaction that has a `BEGIN` in the WAL but no `COMMIT` or `ABORT` record, all operations are replayed in reverse order using `_undo_single()`. This handles the scenario where a crash occurred mid-transaction, leaving dirty writes in the in-memory B+ Tree. The undo restores the pre-transaction state exactly as a normal rollback would.

After both phases complete, the fully recovered state is written back to `database.dat` via `save_to_disk()` and the WAL is cleared. This compaction step ensures that the next startup begins from a clean baseline and does not re-process stale log entries.

Snapshots are taken only when no other transaction is active. This is enforced in both `commit()` and `rollback()` via a global lock and a check on `len(self._active) == 0`. The rationale is that `save_to_disk()` serialises the live in-memory state, so snapshotting while another transaction holds dirty writes would capture uncommitted data which would violate isolation in the snapshot file even if not in the live tree.

1.2.4 Multi-User Conflict Handling

Concurrent access is managed by the `LockManager`, which maintains a dictionary mapping resource IDs of the form `"db_name.table_name"` to the transaction ID that currently holds the lock.

Exclusive Table-Level Locking

All locks are exclusive: only one transaction may hold a lock on a given table at a time. Every read, insert, update, and delete acquires the lock on the relevant table before accessing the B+ Tree. Because the B+ Tree itself is not thread-safe, this table-level granularity is the minimum necessary to prevent structural corruption under concurrent access.

Locks are acquired using a spin-wait with a configurable timeout (default 5 seconds). The acquiring transaction polls every 50 milliseconds; if the lock is not available within the deadline, a `DeadlockError` is raised and the transaction is aborted. While this timeout-

based approach does not detect true deadlock cycles analytically, it prevents indefinite blocking and provides a practical safety net for the implemented concurrency level.

Locks are re-entrant within the same transaction: if a transaction already owns a lock on a table, a subsequent `acquire()` call for the same resource returns immediately without blocking. This is essential for multi-step operations such as FK checks, where the same parent table may be locked both during the forward FK check and again as the target of an insert.

Lock Release

All locks held by a transaction are released atomically via `LockManager.release_all(tx_id)` at the end of both `commit()` and `rollback()`. Releasing all locks only at transaction end is equivalent to strict two-phase locking, which guarantees serializability and prevents cascading rollbacks. a transaction holds all its locks until it has fully committed or fully rolled back, so no other transaction can read or write a value that has been modified but not yet finalised.

WAL Thread Safety

The `WALManager` protects all append operations with its own `threading.Lock()`, independent of the table locks. This ensures that log entries from concurrent transactions are not interleaved at the byte level, preserving the integrity of the JSON log format used for recovery.

The `TransactionManager` also maintains a `_global_lock` to serialise mutations to the `_active` dictionary (the map of currently running transactions). This prevents race conditions when two threads simultaneously begin or commit transactions.

1.3 Testing

All tests are implemented as cells in `acid.ipynb` using the `sports_db` database. Each test ends with a Python `assert` statement to programmatically verify the expected outcome. The schema used across all tests is described in Table 1.1.

Table	Primary Key	Constraints & Relationships
member	MemberID	CHECK('x > 0'), UNIQUE Email, NOT NULL Email
sport	SportID	UNIQUE SportName, CHECK('x is None or x > 0')
team	TeamID	FK → member (CoachID, CaptainID), FK → sport (SportID)
equipment	EquipmentID	CHECK('x >= 0') on TotalQuantity, FK → sport (SportID)

Table 1.1: Schema used for ACID validation tests.

1.3.1 Atomicity Tests

Test A1 : Multi-table COMMIT (success path). A single transaction spanning three relations was used to set up a new Cricket initiative: (1) update Cricket Bat quantity from 10 to 12, (2) promote Harsh Pandey (Member 11) from Player to Coach, (3) insert Team 16 referencing Harsh. All three operations succeeded and `tm.commit()` was called. Post-commit assertions confirmed that the bat quantity read 12, Harsh's role was Coach, and Team 16 was present in the B+ Tree.

Test A2 : Multi-table failure and automatic rollback. The same three-step pattern was used to trigger a mid-transaction failure: (1) deduct five Basketballs (quantity 12 → 7), (2) promote Aarav Sharma to Coach, (3) attempt to insert a team with a non-existent `CoachID = 9999`. Step 3 raised a `ConstraintError` (FK violation), which caused `rollback()` to reverse steps 1 and 2. Post-rollback assertions confirmed that Basketball quantity remained at 12 and Aarav's role remained Player.

```
BEFORE: Basketball Quantity: 12 | Aarav Role: Player
[TxManager] BEGIN  tx=9f0c5b9b
[TxManager] ABORT  tx=9f0c5b9b
ERROR: Foreign Key Violation: 'CoachID'=9999 does not exist in 'member'
AFTER ROLLBACK: Basketball Quantity: 12 | Aarav Role: Player
```

Test A3 : Primary Key shift rollback. A transaction first successfully shifted Member 10's PK to 999, then failed on a subsequent insert of a sport with `MaxPlayersPerTeam = -5` (CHECK violation). The undo log deleted key 999 and restored key 10. Post-rollback assertions confirmed that Member 10 existed and Member 999 did not.

```
AFTER ROLLBACK: Member 10 exists: True
```

```
AFTER ROLLBACK: Member 999 exists: False
PK Shift Rollback verified
```

1.3.2 Consistency Tests

Test C1 : CHECK constraint enforcement. An attempt was made to reduce Volleyball stock (Equipment 9, quantity 10) by 50, which would yield −40. The `validate_record()` method evaluated `'x >= 0'` and rejected the update. The transaction was aborted and the final stock remained at 10.

```
[BLOCKED] Update failed in 'equipment':
          Constraint Error: CHECK failed for 'TotalQuantity'='-40'
Final Volleyball Quantity remains: 10
```

Test C2 : Referential integrity: blocked delete/update of referenced parent. A transaction attempted to delete/update Sport 1 (Athletics), which is referenced by three team records. The reverse FK scan raised a `ConstraintError` and the transaction was aborted. Sport 1 remained present and untouched.

```
[BLOCKED DELETE] Foreign Key Violation: Cannot delete '1' from 'sport'.
                 It is still referenced by 'team'.
```

For update:

```
[BLOCKED UPDATE] Foreign Key Violation: Cannot change PK '2' in 'sport'.
                 It is still referenced by 'team'.
Verified: Sport 1 and 2 are safe and cannot be orphaned.
```

1.3.3 Durability Tests

Test D1 : Persist committed data across restart. Member 100 ("Test Athlete") was inserted and committed. A completely fresh `DatabaseManager` was instantiated and a new `TransactionManager` booted against `wal.log`, triggering `load_from_disk()` followed by the WAL recovery phases. The restored member table returned the correct record for key 100, confirming that committed data survives a full restart.

```
[TxManager] BEGIN  tx=5b809fbf
[TxManager] COMMIT tx=5b809fbf
Member 100 committed.
```

```
[Recovery] REDONE 4 committed transaction(s)
Member 100 after restart: Test Athlete
Durability verified
```

Test D2 : WAL UNDO: incomplete transaction after simulated crash. A `BEGIN` and `OP` (INSERT of Equipment 999, "Ghost Item") were written directly to `wal_crash.log` without a `COMMIT` record, and the same record was simultaneously force-inserted into the live B+ Tree to mimic a crash mid-transaction. Booting a `TransactionManager` against this log triggered the UNDO phase: Equipment 999 was deleted and the key list contracted from `[1..15, 999]` back to `[1..15]`.

```
Equipment Keys BEFORE recovery: [1, 2, ..., 15, 999]
[Recovery] UNDOING incomplete tx=crash001 (1 ops)
[Recovery] UNDONE 1 incomplete transaction(s)
Equipment Keys AFTER recovery: [1, 2, ..., 15]
WAL recovery verified
```

Test D3 : WAL REDO: committed data missing from disk. A complete `BEGIN / OP / COMMIT` sequence for inserting Equipment 888 ("Redo Item") was written to `wal_redo.log`, but the record was deliberately absent from the in-memory B+ Tree to simulate a crash between the WAL write and the `save_to_disk()` call. On boot, the REDO phase identified the committed transaction and re-applied the insert. `eq_tbl.get(888)` returned the record, confirming that the REDO phase bridges the durability gap.

```
Equipment 888 exists BEFORE REDO: False
[Recovery] REDONE 1 committed transaction(s)
Equipment 888 exists AFTER REDO: True
WAL REDO recovery verified
```

1.3.4 Isolation Tests

Test I1: Concurrent Table-Level Modifications Two threads simultaneously attempt to deduct from the same Badminton Racket stock (Equipment 2, starting quantity 15). Thread A deducts 5 and holds the lock for 200 ms; Thread B starts 50 ms after A and deducts 3.

Outcome: The lock manager serialised the two transactions. Both committed successfully, and the final stock was $15 - 5 - 3 = 7$, the only correct result. Had no locking been in place, a lost-update anomaly could have produced a final value of 10 or 12 instead.

```
[TxManager] COMMIT tx=a55eeb51   (Thread A)
[TxManager] COMMIT tx=5092eb94   (Thread B)
Racket stock: started=15, expected=7, actual=7
Isolation verified
```

1.4 Observations and Limitations

1.4.1 Observations

The implementation successfully demonstrates all four ACID properties on a realistic relational schema. Several design decisions are worth noting.

The WAL-first, snapshot-second protocol correctly handles the two most common durability failure modes. Experiment D2 verified the UNDO path and Experiment D3 verified the REDO path independently, giving confidence that both phases of the recovery procedure work correctly in isolation. The idempotency guards in `_redo_single()` mean that re-running recovery on an already-recovered database is safe, which is important given that the WAL is cleared only after recovery completes.

The idle-only checkpoint policy (snapshotting only when `len(_active) == 0`) is a correct, conservative approach to avoiding dirty snapshots. In Experiment I1, the two concurrent threads both commit before the checkpoint fires, ensuring the snapshot captures both changes atomically.

We have successfully tested all aspects and given a detailed description of the process.

1.4.2 Limitations

1. While our current architecture prioritizes data integrity by strictly preventing the accidental deletion of referenced records, we could introduce enhanced referential action protocols like ON DELETE CASCADE and ON DELETE SET NULL options.

2. The LockManager uses a spin-wait with a 5- second timeout for deadlock detection. We could implement an algorithm such as wait-for graph for completeness.

3. Implementation of some additional functionalities present in SQL.